

postdetector

August 23, 2012
9:12

Contents

1	Compute detector signals in a post-processing mode	1
2	Compute signal	11
3	INDEX	21

1 Compute detector signals in a post-processing mode

This version has been generalized to a large degree, although it's only been applied to Gas Puff Imaging (GPI) experiments. Namely, it handles both D₂ (with photons coming from D, D₂, and D₂⁺) and He puffed gases. The only other problem specific information needed is the camera resolution (which happens to be the same for the applications considered so far) and the target plane. The data for the latter are hardcoded here with the specific instances identified with the *APP* macro. In principle, these could be moved to an input file. However, the camera resolution must remain in macro statements since the corresponding parameters appear in dimension statements. Note that all of the application specific information must match that in the corresponding *gpi_views* routine (see, e.g., [gpicaera.web](#)).

In addition to the total emission rate (i.e., the quantity to be compared with the experimentally observed emission), various other quantities are computed to facilitate checking the simulated camera geometry (e.g., the “target” *R* and *Z*) and to facilitate interpretation of the results (e.g., the “effective densities”). Regarding the latter, no single quantity has been identified as the most useful. For examples, see D. P. Stotler et al., Contrib. Plasma Phys. **44**, 294 (2004) and J. R. Myra et al., Phys. Plasmas **18**, 012305 (2011).

These quantities and the corresponding file (or variable) names are:

Name	Description	Symbol
emtt	Total emission rate (W/(m ² sr))	S
empt	Emission rate by species <i>p</i> (W/(m ² sr))	S_p
tarr	Target major radius (m)	$R(j_i)$
tarz	Target vertical coordinate (m)	$Z(j_i)$
tate	Target electron temperature (eV)	$T_e(j_i)$
tane	Target electron density (m ⁻³)	$n_e(j_i)$
fwpa	Emission rate per atom (W)	$F(j_i) = S(j_i)/n_1(j_i)$
nef1	Effective test density 1 (m ⁻² sr ⁻¹)	$S/F(j_i)$
favg	Average emission rate per atom (W)	$\langle F \rangle \equiv \langle S \rangle / \langle n_1 \rangle$
nef3	Effective test density 3 (m ⁻² sr ⁻¹)	$S/\langle F \rangle$
fwps	Specific emission rate per atom (W)	$S_1/n_1(j_i)$
nsf1	Specific effective test density 1 (m ⁻² sr ⁻¹)	$S_1/F(j_i)$
nsf3	Specific effective test density 3 (m ⁻² sr ⁻¹)	$S_1/\langle F_1 \rangle$
tnsp	Target density of species <i>p</i> (m ⁻³)	$n_p(j_i)$
n2sp	Effective density 2 of species <i>p</i> (m ⁻² sr ⁻¹)	$n_{\text{eff2},p}$
fasp	Average specific emission rate per species <i>p</i> (W)	$\langle F_p \rangle \equiv \langle S_p \rangle / \langle n_p \rangle$

Where:

- “Species *p*” refers to the *p*th species in the problem species list.

- If the “emission rate by species” tally is present, the emission rate and derived quantities can be divided into contributions from the emitting species (e.g., D, D_2^+ , D_2). One of these is designated as the “principal” emitter (“p=1” e.g., D).
- The function F is the “atomic physics” function for the principal emitter, i.e., the emission rate per atom evaluated at the local electron density and temperature from the input atomic physics tables (i.e., the density of the excited state relative to ground multiplied by the Einstein coefficient and energy of the transition).
- The “target” quantities are set first by identifying the zone (j_i) along each chord (i) closest to the target plane and then utilizing the corresponding quantity for that zone.
- The “average” quantities (e.g., $\langle S \rangle$) are averaged along a viewing chord over zones within a specified distance (d_{l0} in the code; e.g., 5 cm) of the target plane.
- The second effective density is the emitting particle’s density integrated along the entire viewing chord (i.e., computed just as is the total emission rate).

\$Id: postdetector.web,v 1.4 2011/03/04 18:04:51 dstotler Exp \$

```
"postdetector.f" 1 ≡
@m FILE 'postdetector.web'
@m MPI_messages 0
```

The main program.

```
"postdetector.f" 1.1 ≡
program postdetector
    implicit_none_f77
    mp_common
    implicit_none_f90
    mp_decls

    mpi_init

    if (mpi_master)
        call readfilenames
        call degas_init    // In degasinit.web

    if (mpi_master) then    /* This is not MPI-aware. Instead, just need to extract the required data
        and broadcast it to the slaves. */
        call nc_read_output    // In pmimatread.web
        call fill_views_master
    else
        call fill_views_slave
    end if

    mpi_end

    stop
end
```

⟨ Functions and Subroutines 1.2 ⟩

The master routine for evaluating the views.

```
"postdetector.f" 1.2 ≡
  @m NSTX 0    // Macro for application. Could specify different
  @m CMOD_XPT 1    // nx and ny here if needed.
  @m CMOD_MID 2
  @m NSTX_2010 3

  @m APP NSTX

@#if 0
@#if (APP ≡ NSTX_2010)
  @m nx 64
  @m ny 80
@#else
  @m nx 64
  @m ny 64
@#endif
@#else
  @m nx 5
  @m ny 5
@#endif
@m nsd 3    // Maximum number of test species yielding photons

  /* Remove characters causing file name problems */
@m name_clean(s, sp) sp = s
  ind_tmp = index(sp, '(')
  if (ind_tmp > 0)
    sp(ind_tmp : ind_tmp) = '_'
  ind_tmp = index(sp, ')')
  if (ind_tmp > 0)
    sp(ind_tmp : ind_tmp) = '_'
  ind_tmp = index(sp, '|')
  if (ind_tmp > 0)
    sp(ind_tmp : ind_tmp) = '_'
  ind_tmp = index(sp, '+')
  if (ind_tmp > 0)
    sp(ind_tmp : ind_tmp) = 'p'
```

⟨ Functions and Subroutines 1.2 ⟩ ≡

subroutine *fill_views_master*

```
  define_dimen(species_ind, nsd)

  define_varp(em_data, FLOAT, zone_ind)
  define_varp(em_sp_data, FLOAT, species_ind, zone_ind)
  define_varp(test_density, FLOAT, species_ind, zone_ind)
  define_varp(zone_frags, FLOAT, zone_ind)

  implicit_none_f77
  sp_common
  pr_common
  tl_common
  ou_common
  bk_common
  zn_common
```

```

de_common
mp_common
implicit_none_f90

integer zone, test, ix, iy, inverted_iy, algorithm, nviews, nfrag, nparcel, nslaves, j, nstart, num,
      slave, send_data, iview, close_zone, nx_peak, iy_peak, is, total_tally, species_tally, itally, len,
      ns, ind_tmp

integer index_parameters u_index_max, target_map nx,ny, target_map_slave nx,ny
real em_temp, em_rate, halfwidth, signal, a0_p, a1_p, a2_p, dist, min_dist, opt, max_opt, em_avg,
      a0, a1, a2, a3, a3_p, phi, d_l0, d_opt, peak, cy_l, cy_r
real x_array nx, y_array ny, signal_array nx,ny, signal_array_slave nx,ny, temp_array nx,ny,
      target_r nx,ny, target_z nx,ny, target_n nx,ny,nsd, w_per_atom nx,ny, w_sp_per_atom nx,ny,
      n_eff1 nx,ny, n_sp_eff1 nx,ny, n_eff2 nx,ny,nsd, n_eff2_slave nx,ny,nsd, fwpa_avg nx,ny,
      fwpa_avg_slave nx,ny, n_eff3 nx,ny, n_sp_eff3 nx,ny, target_te nx,ny, target_ne nx,ny,
      signal_sp_array nx,ny,nsd, fwpa_sp_avg nx,ny,nsd, signal_sp_array_slave nx,ny,nsd,
      fwpa_sp_avg_slave nx,ny,nsd, profile ny, n_eff2_sum nsd, signal_sp nsd, n_avg nsd, em_sp_avg nsd

character* sp_sy_len clean_sy
character* s

vc_decl(points de_view_start:de_view_end)
external extract_output_datum, gpi_views
real extract_output_datum

declare_varp(em_data)
declare_varp(em_sp_data)
declare_varp(test_density)
declare_varp(zone_frgs)

⟨ Memory allocation interface 0 ⟩
st_decls
vc_decls
mp_decls

var_alloc(em_data)
var_alloc(em_sp_data)
var_alloc(test_density)
var_alloc(zone_frgs)

do zone = 1, zn_num
  em_data zone = zero
  do is = 1, nsd
    em_sp_data zone,is = zero
    test_density zone,is = zero
  end do
end do

total_tally = int_undef
species_tally = int_undef
do itally = 1, tl_num
  len = string_length(tally_name itally)
  if (tally_name itally SP(len - 12 :) ≡ 'emission_rate')
    total_tally = itally
  if (tally_name itally SP(len - 23 :) ≡ 'emission_rate_by_species')
    species_tally = itally
  end do
assert(tl_check(total_tally))

```

```

if (tl_check(species_tally)) then
    ns = pr_test_num - 1    // Skip the “geometry” test species
else
    ns = 1    // Only interested in one species; assume it’s the first.
end if

em_temp = zero
do zone = 1, zn_num
    if (zn_type(zone)  $\equiv$  zn_plasma) then
        index_parameterstl_index_zone = zone
        em_rate = extract_output_datum(index_parameters, 1, out_post_all, o_mean, tally_nametotal_tally)
        em_datazone = em_rate
        em_temp += em_rate * zn_volume(zone)
        do test = 2, pr_test_num    // Skip the “geometry” test species
            index_parameterstl_index_test = test
            is = test - 1
            test_densityzone,is = extract_output_datum(index_parameters, 1, out_post_all, o_mean,
                'neutral_density')
            if (tl_check(species_tally)) then
                em_rate = extract_output_datum(index_parameters, 1, out_post_all, o_mean,
                    tally_namespecies_tally)
                em_sp_datazone,is = em_rate
            end if
        end do
    end if
end do

@#if MPI
    call MPI_bcast(em_data, zn_num, mpi_real, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)
    call MPI_bcast(em_sp_data, zn_num*nsd, mpi_real, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)
    call MPI_bcast(test_density, zn_num*nsd, mpi_real, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)
    call MPI_bcast(ns, 1, mpi_int, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)
@#endif

    if (em_temp > zero) then
        nviews = nx * ny    /* Copied this logic directly from do_flights_master. */
    @#if MPI
        nslaves = mpi_size - 1
    @#else
        nslaves = 0
    @#endif
    if
        nfrag = max(nviews / max(100, 10 * nslaves), 1)    // Size of each chunk
        nparcel = (nviews + nfrag - 1) / nfrag    // Number of chunks
    @#if MPI
        do j = 0, min(nparcel, nslaves) - 1    // Send out the initial chunks
            nstart = j * nfrag
            num = min(nfrag, nviews - nstart)
            slave = j + 1
            @#if MPI_messages
                write (stderr, *) 'Send_initial_batch_of_', num, '_to_slave_', slave
            @#endif
            call slave_send_views(slave, nstart, num)
        end do
    @#endif

```

```

do j = min(nparcel, nslaves), nparcel - 1
    // Loop reading results and making additional requests
    nstart = j * nfrag
    num = min(nfrag, nviews - nstart)
    @#if MPI
        if (nslaves > 0) then
            call slave_receive_flag(slave)
        @#if MPI_messages
            write(stderr, *) 'Slave_', slave, '_is_done_with_a_batch'
            write(stderr, *) 'Sending_', nfrag, '_more_to_slave_', slave
        @#endif
        send_data = FALSE
        call slave_send_flag(slave, send_data)
        call slave_send_views(slave, nstart, num)
    else
        assert('Why_is_nslaves_=0_in_an_MPI_run?' == '')
    end if
    @#else
        < Compute Signal 2 >
    @#endif
end do

@#if MPI
do j = 0, min(nparcel, nslaves) - 1    // Retrieve data from slaves this time
    call slave_receive_flag(slave)
    send_data = TRUE
    call slave_send_flag(slave, send_data)
    call slave_receive_data(slave, signal_array_slave, target_map_slave, fupa_avg_slave,
        n_eff2_slave, signal_sp_array_slave, fupa_sp_avg_slave)

    do iy = 1, ny
        do ix = 1, nx
            if (signal_array_slaveix,iy ≠ real_unused) then
                signal_arrayix,iy = signal_array_slaveix,iy
                assert(target_map_slaveix,iy ≠ int_unused)
                target_mapix,iy = target_map_slaveix,iy
                fupa_avgix,iy = fupa_avg_slaveix,iy
                do is = 1, ns
                    assert(n_eff2_slaveix,iy,is ≠ real_unused)
                    n_eff2ix,iy,is = n_eff2_slaveix,iy,is
                    signal_sp_arrayix,iy,is = signal_sp_array_slaveix,iy,is
                    fupa_sp_avgix,iy,is = fupa_sp_avg_slaveix,iy,is
                end do
            end if
        end do
    end do

end do

do j = 0, nslaves - 1    // Kill off slaves
    slave = j + 1
    call slave_send_views(slave, 0, 0)
end do
@#endif

```

```

do ix = 1, nx
  x_array_ix = areal(ix - 1)
end do

do iy = 1, ny
  @#if (APP == NSTX_2010)
    y_array_iy = areal(iy - 1)
  @#else
    inverted_iy = ny - iy + 1
    y_array_inverted_iy = areal(iy - 1)
  @#endif
end do

call wrdatap(signal_array, x_array, y_array, 1, nx, 1, ny, 'total_emission_rate',
  'W/(m2_sr)', 'emtt', 1, 1, 'E11.3')

if (ns > 1) then
  do is = 1, ns
    do iy = 1, ny
      do ix = 1, nx
        temp_array_ix,iy = signal_sp_array_ix,iy,is
      end do
    end do
    name_clean(sp_sy(pr_test(is + 1)), clean_sy)    // see geomtesta.web
    assert(ns < 10)
    write(s, '(i1)') is
    call wrdatap(temp_array, x_array, y_array, 1, nx, 1, ny,
      'emission_rate_by_' || trim(clean_sy), 'W/(m2_sr)', 'ems' || s, 1, 1, 'E11.3')
  end do
end if

@#if 0
  nx_peak = 33    // I.e., at cx = 32
  iy_peak = 0
  peak = zero
  do iy = 1, ny
    profile_iy = signal_array_nx_peak,iy
    if (profile_iy > peak) then
      peak = profile_iy
      iy_peak = iy
    end if
  end do
  assert(peak > zero)
  assert(iy_peak > 0)

  assert(half * peak > profile_1)
  cy_l = zero
  cy_r = zero
  do iy = 1, ny
    if ((cy_l == zero) ^ (profile_iy > half * peak)) then
      cy_l = y_array_iy-1 - (half * peak - profile_iy-1) / (profile_iy - profile_iy-1)
    end if
    if ((cy_r == 0) ^ (cy_l > zero) ^ (profile_iy < half * peak)) then
      cy_r = y_array_iy-1 - (half * peak - profile_iy-1) / (profile_iy - profile_iy-1)
    end if
  end do
end if

```

```

end do
open(unit = diskout, file = 'detectorprofile.txt', status = 'unknown')
write(diskout, *) 'At_Cx=', x_arraynx-peak, ', peak=', peak, ', at_cy=', y_arrayiy-peak
write(diskout, *) 'Half_peak_at_cy=', cy-l, ', and ', cy-r, ', FWHM=', cy-r - cy-l
write(diskout, *) 'iy_profile'
do iy = 1, ny
  write(diskout, *) y_arrayiy, profileiy
end do
close(unit = diskout)
@#endif
do iy = 1, ny
  do ix = 1, nx
    if (zn_check(target_mapix,iy)) then
      zone = target_mapix,iy
      target_rix,iy = sqrt(zone_centerzone,12 + zone_centerzone,22)
      target_zix,iy = zone_centerzone,3
      target_teix,iy = bk_temp(1, zone) / electron_charge
      target_neix,iy = bk_n(1, zone)
      do is = 1, ns
        target_nix,iy,is = test_densityzone,is
      end do
      if (test_densityzone,1 > zero) then
        w_per_atomix,iy = em_datazone / test_densityzone,1 /* This is just the emission by the
          principal species in the multi-species case. Skip the other permutations. */
        w_sp_per_atomix,iy = em_sp_datazone,1 / test_densityzone,1
      else
        w_per_atomix,iy = zero
        w_sp_per_atomix,iy = zero
      end if
    else
      target_rix,iy = zero
      target_zix,iy = zero
      target_teix,iy = zero
      target_neix,iy = zero
      do is = 1, ns
        target_nix,iy,is = zero
      end do
      w_per_atomix,iy = zero
      w_sp_per_atomix,iy = zero
    end if
    if (w_per_atomix,iy > zero) then
      n_eff1ix,iy = signal_arrayix,iy / w_per_atomix,iy
    else
      n_eff1ix,iy = zero
    end if
    if (w_sp_per_atomix,iy > zero) then
      n_sp_eff1ix,iy = signal_sp_arrayix,iy,1 / w_sp_per_atomix,iy
    else
      n_sp_eff1ix,iy = zero
    end if
    if (fwpa_avgix,iy > zero) then
      n_eff3ix,iy = signal_arrayix,iy / fwpa_avgix,iy
    end if
  end do
end do

```

```

    end if
    if (fwpa_sp_avgix,iy,1 > zero) then
        n_sp_eff3ix,iy = signal_sp_arrayix,iy,1 / fwpa_sp_avgix,iy,1
    else
        n_sp_eff3ix,iy = zero
    end if
end do
end do
end if

call wrdatap(target_r, x_array, y_array, 1, nx, 1, ny, 'target_major_radius', 'm', 'tarr', 1,
1, 'E11.3')
call wrdatap(target_z, x_array, y_array, 1, nx, 1, ny, 'target_vertical_coord', 'm', 'tarz',
1, 1, 'E11.3')
call wrdatap(target_te, x_array, y_array, 1, nx, 1, ny, 'target_electron_temperatu\
re', 'eV', 'tate', 1, 1, 'E11.3')
call wrdatap(target_ne, x_array, y_array, 1, nx, 1, ny, 'target_electron_density', 'm^3',
'tane', 1, 1, 'E11.3')
call wrdatap(w_per_atom, x_array, y_array, 1, nx, 1, ny, 'emission_rate_per_atom', 'W',
'fwpa', 1, 1, 'E11.3')
call wrdatap(n_eff1, x_array, y_array, 1, nx, 1, ny, 'effective_test_density_1',
'm^2_sr^-1', 'nef1', 1, 1, 'E11.3')
call wrdatap(fwpa_avg, x_array, y_array, 1, nx, 1, ny, 'avg_emission_rate_per_ato\
m', 'W', 'favg', 1, 1, 'E11.3')
call wrdatap(n_eff3, x_array, y_array, 1, nx, 1, ny, 'effective_test_density_3',
'm^2_sr^-1', 'nef3', 1, 1, 'E11.3')

if (tl_check(species_tally)) then
    call wrdatap(w_sp_per_atom, x_array, y_array, 1, nx, 1, ny, 'specific_emission_rate_pe\
r_atom', 'W', 'fwps', 1, 1, 'E11.3')
    call wrdatap(n_sp_eff1, x_array, y_array, 1, nx, 1, ny, 'specific_effective_test_d\
ensity_1', 'm^2_sr^-1', 'nsf1', 1, 1, 'E11.3')
    call wrdatap(n_sp_eff3, x_array, y_array, 1, nx, 1, ny, 'specific_effective_test_d\
ensity_3', 'm^2_sr^-1', 'nsf3', 1, 1, 'E11.3')
end if

do is = 1, ns
    name_clean(sp_sy(pr_test(is + 1)), clean_sy)
    write(s, '(i1)') is
    do iy = 1, ny
        do ix = 1, nx
            temp_arrayix,iy = target_nix,iy,is
        end do
    end do
    call wrdatap(temp_array, x_array, y_array, 1, nx, 1, ny,
'target_density_of_' || trim(clean_sy), 'm^3', 'tns' || s, 1, 1, 'E11.3')
    do iy = 1, ny
        do ix = 1, nx
            temp_arrayix,iy = n_eff2ix,iy,is
        end do
    end do
    call wrdatap(temp_array, x_array, y_array, 1, nx, 1, ny,
'effective_density_2_of_' || trim(clean_sy), 'm^2_sr^-1', 'n2s' || s, 1, 1, 'E11.3')
    do iy = 1, ny

```

```

    do ix = 1, nx
      temp_arrayix,iy = fwpa-sp-avgix,iy,is
    end do
  end do
  call wrdatap(temp_array, x_array, y_array, 1, nx, 1, ny, 'avg_specific_emission_rat\
    e_per_' || trim(clean_sy), 'W', 'fas' || s, 1, 1, 'E11.3')
end do

var_free(em_data)
var_free(em_sp_data)
var_free(test_density)
var_free(zone_frgs)

return
end

```

See also sections 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, and 2.7.

This code is used in section 1.1.

2 Compute signal

This code is repeated in the master and slave *fill_views* routines.

```

⟨ Compute Signal 2 ⟩ ≡
    /* Specific parameters for this application */
    @#if (APP ≡ NSTX)
        a0 = -const(3.60752)    // Coefficients of the camera's "target plane"
        a1 = const(2.11865)
        a2 = const(3.17544)
        a3 = -one
        phi = const(6.146, 1) * PI / const(1.8, 2)    // Toroidal shift to my system
        d_l0 = const(5., -2)    // Range for averaging about target plane
        d_opt = const(2.5, -1)  // Range for searching for a close zone
    @#elif (APP ≡ CMOD.MID)
        a0 = -sin(const(-18.48) * PI / const(1.8, 2))    // A vertical plane at this
        a1 = cos(const(-18.48) * PI / const(1.8, 2))    // toroidal angle
        a2 = zero
        a3 = zero
        phi = zero    // No additional toroidal shift needed
        d_l0 = const(5., -2)
        d_opt = const(3.0, -1)
    @#elif (APP ≡ CMOD.XPT)
        a0 = -sin(const(33.0) * PI / const(1.8, 2))    // Again, a vertical plane
        a1 = cos(const(33.0) * PI / const(1.8, 2))
        a2 = zero
        a3 = zero
        phi = zero
        d_l0 = const(5., -2)
        d_opt = const(3.0, -1)
    @#elif (APP ≡ NSTX_2010)
        a0 = -const(3.58653)    // Coefficients of the camera's "target plane"
        a1 = const(2.08796)
        a2 = const(3.18466)
        a3 = -one
        phi = const(6.146, 1) * PI / const(1.8, 2)    // Toroidal shift to my system
        d_l0 = const(5., -2)    // Range for averaging about target plane
        d_opt = const(2.5, -1)  // Range for searching for a close zone
    @#else
        assert('Target_plane_parameters_not_specified' ≡ '␣')
    @#endif
    /* Rotate Ricky's "target plane" to my coordinate system. The "p" denotes "prime". */
    a0_p = a0 * cos(phi) + a1 * sin(phi)
    a1_p = a1 * cos(phi) - a0 * sin(phi)
    a2_p = a2
    a3_p = a3
    do iview = nstart, nstart + num - 1
        if (mod(iview + 1, nx) ≠ 0) then
            iy = (iview + 1) / nx + 1
            ix = (iview + 1) - (iy - 1) * nx
        else
            iy = (iview + 1) / nx

```

```

    ix = nx
end if
call gpi_views(ix, nx, iy, ny, vc_args(pointsde-view-start), halfwidth, algorithm, zone_frags)
signal = zero
em_avg = zero
do is = 1, ns
    n_eff2_sumis = zero
    signal_spis = zero
    n_avgis = zero
    em_sp_avgis = zero
end do
min_dist = const(1., 16)
max_opt = const(1., -16)
close_zone = int_undef
do zone = 1, zn_num
    signal += em_datazone * zone_fragszone * zn_volume(zone)
    do is = 1, ns
        n_eff2_sumis += test_densityzone,is * zone_fragszone * zn_volume(zone)
        signal_spis += em_sp_datazone,is * zone_fragszone * zn_volume(zone)
    end do /* Hack to distill out zone closest to target plane. */
    dist = abs(a0_p * zone_centerzone,1 + a1_p * zone_centerzone,2 + a2_p * zone_centerzone,3 + a3_p) /
        sqrt(a0_p2 + a1_p2 + a2_p2)
    if (dist ≠ zero) then
        opt = zone_fragszone * zn_volume(zone) / dist
    else /* In the C-Mod problems, can actually get dist = 0. Use a finite length for computing
        opt to distinguish these in-target-plane zones. */
        opt = zone_fragszone * zn_volume(zone) / const(1., -8)
    end if
    @#if 0
        if ((zone_fragszone > zero) ∧ (dist < min_dist)) then
            @#endif
                if ((opt > max_opt) ∧ (dist ≤ d_opt)) then
                    max_opt = opt
                    min_dist = dist
                    close_zone = zone
                end if /* Sums restricted to region around target plane. */
                if (dist ≤ d_l0) then
                    em_avg += em_datazone * zone_fragszone * zn_volume(zone)
                    do is = 1, ns
                        n_avgis += test_densityzone,is * zone_fragszone * zn_volume(zone)
                        em_sp_avgis += em_sp_datazone,is * zone_fragszone * zn_volume(zone)
                    end do
                end if
            end do
        @#if (APP ≡ NSTX_2010)
            inverted_iy = iy
        @#else
            inverted_iy = ny - iy + 1
        @#endif
        /* Have a case now in which a chord does not reach the target plane. Will let close_zone remain
        int_undef and then assign zero values to the associated target parameters. */
    @#if 0

```

```

    assert(close_zone ≠ int_undef)
    assert(min_dist < const(1., 16))
@#endif
    target_mapix,inverted_iy = close_zone
    signal_arrayix,inverted_iy = signal
    if (n_avg1 > zero) then
        fwpa_avgix,inverted_iy = em_avg / n_avg1
    else
        fwpa_avgix,inverted_iy = zero
    end if
    do is = 1, ns
        n_eff2ix,inverted_iy,is = n_eff2_sumis
        signal_sp_arrayix,inverted_iy,is = signal_spis
        if (n_avgis > zero) then
            fwpa_sp_avgix,inverted_iy,is = em_sp_avgis / n_avgis
        else
            fwpa_sp_avgix,inverted_iy,is = zero
        end if
    end do
end do

```

This code is used in sections 1.2 and 2.1.

Slave version for filling views.

⟨ Functions and Subroutines 1.2 ⟩ +=

@#if MPI

subroutine *fill_views_slave*

define_varp(em_data, FLOAT, zone_ind)

define_varp(em_sp_data, FLOAT, species_ind, zone_ind)

define_varp(test_density, FLOAT, species_ind, zone_ind)

define_varp(zone_frgs, FLOAT, zone_ind)

implicit_none_f77

mp_common

zn_common

implicit_none_f90

integer *iy, ix, tag, num, nstart, iview, is, ns, algorithm, close_zone, zone, inverted_iy, done_flag, send_data*

integer *target_map*_{*nx,ny*}

real *halfwidth, signal, a0_p, a1_p, a2_p, dist, min_dist, a0, a1, a2, a3, phi, d_l0, d_opt, a3_p, em_avg, opt, max_opt*

real *signal_array*_{*nx,ny*}, *n_eff2*_{*nx,ny,nsd*}, *fwpa_avg*_{*nx,ny*}, *signal_sp_array*_{*nx,ny,nsd*}, *fwpa_sp_avg*_{*nx,ny,nsd*}, *n_eff2_sum*_{*nsd*}, *signal_sp*_{*nsd*}, *n_avg*_{*nsd*}, *em_sp_avg*_{*nsd*}

*vc_decl(points*_{*de_view_start:de_view_end*}*)*

declare_varp(em_data)

declare_varp(em_sp_data)

declare_varp(test_density)

declare_varp(zone_frgs)

⟨ Memory allocation interface 0 ⟩

vc_decls

mp_decls

var_alloc(em_data)

var_alloc(em_sp_data)

var_alloc(test_density)

var_alloc(zone_frgs)

call *MPI_bcast(em_data, zn_num, mpi_real, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)*

call *MPI_bcast(em_sp_data, zn_num*nsd, mpi_real, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)*

call *MPI_bcast(test_density, zn_num*nsd, mpi_real, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)*

call *MPI_bcast(ns, 1, mpi_int, mpi_degas2_root, MPI_COMM_WORLD, mpi_err)*

do *iy = 1, ny*

do *ix = 1, nx*

*signal_array*_{*ix,iy*} = *real_unused*

*target_map*_{*ix,iy*} = *int_unused*

*fwpa_avg*_{*ix,iy*} = *real_unused*

do *is = 1, ns*

*n_eff2*_{*ix,iy,is*} = *real_unused*

*signal_sp_array*_{*ix,iy,is*} = *real_unused*

*fwpa_sp_avg*_{*ix,iy,is*} = *real_unused*

end do

end do

end do

```

loop: continue
  tag = 100
  call MPI_recv(num, 1, mpi_int, mpi_degas2_root, tag, MPI_COMM_WORLD, mpi_status, mpi_err)
  if (num == 0)
    goto break
  tag++
  call MPI_recv(nstart, 1, mpi_int, mpi_degas2_root, tag, MPI_COMM_WORLD, mpi_status, mpi_err)
  < Compute Signal 2 >

  done_flag = TRUE
  tag = 200
  call MPI_send(done_flag, 1, mpi_int, mpi_degas2_root, tag, MPI_COMM_WORLD, mpi_err)

  tag = 300
  call MPI_recv(send_data, 1, mpi_int, mpi_degas2_root, tag, MPI_COMM_WORLD, mpi_status,
    mpi_err)
  if (send_data == FALSE) then
    goto loop
  else if (send_data == TRUE) then
    tag = 400

    call MPI_send(signal_array, nx * ny, mpi_real, mpi_degas2_root, tag, MPI_COMM_WORLD,
      mpi_err)
    tag = 401
    call MPI_send(target_map, nx * ny, mpi_int, mpi_degas2_root, tag, MPI_COMM_WORLD, mpi_err)
    tag = 402
    call MPI_send(fwpa_avg, nx * ny, mpi_real, mpi_degas2_root, tag, MPI_COMM_WORLD, mpi_err)
    tag = 403
    call MPI_send(n_eff2, nx * ny * nsd, mpi_real, mpi_degas2_root, tag, MPI_COMM_WORLD, mpi_err)
    tag = 404
    call MPI_send(signal_sp_array, nx * ny * nsd, mpi_real, mpi_degas2_root, tag,
      MPI_COMM_WORLD, mpi_err)
    tag = 405
    call MPI_send(fwpa_sp_avg, nx * ny * nsd, mpi_real, mpi_degas2_root, tag,
      MPI_COMM_WORLD, mpi_err)
    do iy = 1, ny
      do ix = 1, nx
        signal_array_ix,iy = real_unused
        target_map_ix,iy = int_unused
        fwpa_avg_ix,iy = real_unused
        do is = 1, ns
          n_eff2_ix,iy,is = real_unused
          signal_sp_array_ix,iy,is = real_unused
          fwpa_sp_avg_ix,iy,is = real_unused
        end do
      end do
    end do
    goto loop
  else
    assert('Illegal_value_of_send_data' == '_')
  end if
break: continue

```

```

    var_free(em_data)
    var_free(em_sp_data)
    var_free(test_density)
    var_free(zone_fragments)
    return
end
@#else
    subroutine fill_views_slave    // dummy
    return
end
@#endif

```

Send data to slave.

⟨ Functions and Subroutines 1.2 ⟩ +≡

```

@#if MPI
    subroutine slave_send_views(slave, nstart, num)
        implicit none_f77
        mp_common
        implicit none_f90
        mp_decls
        integer slave, nstart, num
        integer tag

        ⟨ Memory allocation interface 0 ⟩

        tag = 100
        call MPI_send(num, 1, mpi_int, slave, tag, MPI_COMM_WORLD, mpi_err)
        if (num ≡ 0)
            goto break
        tag++
        call MPI_send(nstart, 1, mpi_int, slave, tag, MPI_COMM_WORLD, mpi_err)

break: continue
    return
end
@#endif

```

Receive notice from a slave indicating that it's done with current batch.

⟨ Functions and Subroutines 1.2 ⟩ +≡

```
@#if MPI
  subroutine slave_receive_flag(slave)
    implicit none f77
    mp_common
    implicit none f90
    mp_decls
    integer slave
    integer tag, done_flag

    ⟨ Memory allocation interface 0 ⟩

    tag = 200
    call MPI_recv(done_flag, 1, mpi_int, MPI_ANY_SOURCE, tag, MPI_COMM_WORLD, mpi_status,
                  mpi_err)
    slave = mpi_status(MPI_SOURCE)
    assert(done_flag ≡ TRUE)
    return
  end
@#endif
```

Send a flag to a slave instructing it to send back data from its flights.

⟨ Functions and Subroutines 1.2 ⟩ +≡

```
@#if MPI
  subroutine slave_send_flag(slave, send_data)
    implicit none f77
    mp_common
    implicit none f90
    mp_decls
    integer slave, send_data    // Input
    integer tag                // Local

    ⟨ Memory allocation interface 0 ⟩

    tag = 300
    call MPI_send(send_data, 1, mpi_int, slave, tag, MPI_COMM_WORLD, mpi_err)

    return
  end
@#endif
```

Receive data from flights run by a slave.

⟨ Functions and Subroutines 1.2 ⟩ +≡

@#if MPI

```
subroutine slave_receive_data(slave, signal_array, target_map, fwpa_avg, n_eff2, signal_sp_array,
    fwpa_sp_avg)
    implicit none f77
    mp_common
    implicit none f90
    mp_decls
    integer slave
    integer target_map nx,ny
    real signal_array nx,ny, fwpa_avg nx,ny, n_eff2 nx,ny,nsd, signal_sp_array nx,ny,nsd,
        fwpa_sp_avg nx,ny,nsd
    integer tag
```

⟨ Memory allocation interface 0 ⟩

```
tag = 400
```

```
call MPI_recv(signal_array, nx * ny, mpi_real, MPLANY_SOURCE, tag, MPLCOMM_WORLD,
    mpi_status, mpi_err)
```

```
tag = 401
```

```
call MPI_recv(target_map, nx * ny, mpi_int, MPLANY_SOURCE, tag, MPLCOMM_WORLD,
    mpi_status, mpi_err)
```

```
tag = 402
```

```
call MPI_recv(fwpa_avg, nx * ny, mpi_real, MPLANY_SOURCE, tag, MPLCOMM_WORLD,
    mpi_status, mpi_err)
```

```
tag = 403
```

```
call MPI_recv(n_eff2, nx * ny * nsd, mpi_real, MPLANY_SOURCE, tag, MPLCOMM_WORLD,
    mpi_status, mpi_err)
```

```
tag = 404
```

```
call MPI_recv(signal_sp_array, nx * ny * nsd, mpi_real, MPLANY_SOURCE, tag,
    MPLCOMM_WORLD, mpi_err)
```

```
tag = 405
```

```
call MPI_recv(fwpa_sp_avg, nx * ny * nsd, mpi_real, MPLANY_SOURCE, tag,
    MPLCOMM_WORLD, mpi_err)
```

```
return
```

```
end
```

@#endif

Rearrange 2D array prior to writing. This configuration is specific to the 2004 NSTX Gas Puff Imaging experiments.

⟨ Functions and Subroutines 1.2 ⟩ +=

```

subroutine wrdatap(pb2dat, xdat, ydat, kxmin, kx, kymin, ky, clabel, cunits, cfilrt, kcount,
    kfilno, cformt)
    implicit_none_f77
    implicit_none_f90

    integer kcount, kfilno, kxmin, kx, ky, kymin    // Input
    character*(*) cunits, clabel
    character*4 cfilrt, cformt
    real xdatkx, ydatky, pb2datkx,ky

    integer ix, iy, ixprime, iyprime    // Local
    real xprimeny, yprimenx, pb2primeny,nx

    assert(kx ≡ nx)
    assert(ky ≡ ny)
    @if (APP ≡ NSTX)
        assert(kx ≡ ky)    /* The required mapping is equivalent to rowflip(transpose(q)) in Transform.
            Namely, swap ix and iy, and then invert the horizontal coordinate. Note that we just assume
            that the dimensions are the same in both directions. */
        do iy = kymin, ky
            do ix = kxmin, kx
                ixprime = iy
                iyprime = kx - ix + 1
                pb2primeixprime,iyprime = pb2datix,iy
            end do
        end do    /* Apply equivalent transformations to the scales. */
        do ix = kxmin, kx
            xprimeix = ydatix
        end do

        do iy = kymin, ky
            ixprime = ky - iy + 1
            yprimeiy = xdatixprime
        end do    /* Call wrdata with the revised arrays */
        call wrdata(pb2prime, xprime, yprime, kxmin, kx, kymin, ky, clabel, cunits, cfilrt, kcount,
            kfilno, cformt)
    @else
        call wrdata(pb2dat, xdat, ydat, kxmin, kx, kymin, ky, clabel, cunits, cfilrt, kcount, kfilno, cformt)
    @endif

    return
end

```

Write 2D array to HDF file. Borrowed this from the H collisional radiative code, irls.

⟨Functions and Subroutines 1.2⟩ +=

```

subroutine wrdata(pb2dat, xdat, ydat, kxmin, kx, kymin, ky, clabel, cunits, cfilrt, kcount,
                 kfilno, cformt)
  implicit_none_f77
  implicit_none_f90

  integer kcount, kfilno, kxmin, kx, ky, kymin    // Input
  character*(*) cunits, clabel
  character*4 cfilrt, cformt
  real xdatkx, ydatky, pb2datkx,ky

  integer icount, iret, ix, iy, irank    // Local
  integer*4 idim2
  character*9 cfile

  single_precision zdatic, zmax, zmin
  single_precision zxdatkx, zydatky, zdatakx*ky

  integer dssdast, dssdims, dssdisc, dssrang, dspdata
  external dssdast, dssdims, dssdisc, dssrang, dspdata, xerrab    /* Set dimension sizes */

  irank = 2
  idim1 = kx - kxmin + 1
  idim2 = ky - kymin + 1    /* Determine maximum and minimum; assign data to 1-D work array. */
  zmax = -const(1., 20)
  zmin = const(1., 20)
  icount = 0

  do iy = kymin, ky
    zydatiy = single(ydatiy)
    do ix = kxmin, kx
      zxdatix = single(xdatix)
      icount++
      zdatic = pb2datix,iy
      zmax = max(zmax, zdatic)
      zmin = min(zmin, zdatic)
      zdataicount = zdatic
    end do
  end do    /* HDF set calls */
  iret = dssdims(irank, idim)
  assert(iret ≡ 0)
  iret = dssdisc(1, idim1, zxdat)
  assert(iret ≡ 0)
  iret = dssdisc(2, idim2, zydat)
  assert(iret ≡ 0)
  iret = dssdast(clabel, cunits, cformt, 'cartesian')
  assert(iret ≡ 0)

  iret = dssrang(zmax, zmin)
  assert(iret ≡ 0)    /* Construct filename */
  write(cfile, '(a4,i2.2,i3.3)') cfilrt, kcount, kfilno    /* Write 2-D data */
  iret = dspdata(cfile || '.hdf', 2, idim, zdata)
  assert(iret ≡ 0)

  return
end

```

3 INDEX

- abs*: 2.
- algorithm*: [1.2](#), 2, [2.1](#).
- APP*: 1, [1.2](#), 2, 2.6.
- areal*: 1.2.
- assert*: 1.2, 2, 2.1, 2.3, 2.6, 2.7.
- a0*: [1.2](#), 2, [2.1](#).
- a0_p*: [1.2](#), 2, [2.1](#).
- a1*: [1.2](#), 2, [2.1](#).
- a1_p*: [1.2](#), 2, [2.1](#).
- a2*: [1.2](#), 2, [2.1](#).
- a2_p*: [1.2](#), 2, [2.1](#).
- a3*: [1.2](#), 2, [2.1](#).
- a3_p*: [1.2](#), 2, [2.1](#).
- bk_common*: 1.2.
- bk_n*: 1.2.
- bk_temp*: 1.2.
- break*: 2.1, 2.2.
- cfile*: [2.7](#).
- cfilrt*: [2.6](#), [2.7](#).
- cformt*: [2.6](#), [2.7](#).
- clabel*: [2.6](#), [2.7](#).
- clean_sy*: [1.2](#).
- close_zone*: [1.2](#), 2, [2.1](#).
- CMOD_MID*: [1.2](#), 2.
- CMOD_XPT*: [1.2](#), 2.
- const*: 2, 2.7.
- cos*: 2.
- cunits*: [2.6](#), [2.7](#).
- cy_l*: [1.2](#).
- cy_r*: [1.2](#).
- d_l0*: 1, [1.2](#), 2, [2.1](#).
- d_opt*: [1.2](#), 2, [2.1](#).
- de_common*: 1.2.
- de_view_end*: 1.2, 2.1.
- de_view_start*: 1.2, 2, 2.1.
- declare_varp*: 1.2, 2.1.
- define_dimen*: 1.2.
- define_varp*: 1.2, 2.1.
- degas_init*: 1.1.
- degasinit*: 1.1.
- diskout*: 1.2.
- dist*: [1.2](#), 2, [2.1](#).
- do_flights_master*: 1.2.
- done_flag*: [2.1](#), [2.3](#).
- dspdata*: [2.7](#).
- dssdast*: [2.7](#).
- dssdims*: [2.7](#).
- dssdisc*: [2.7](#).
- dssrang*: [2.7](#).
- electron_charge*: 1.2.
- em_avg*: [1.2](#), 2, [2.1](#).
- em_data*: 1.2, 2, 2.1.
- em_rate*: [1.2](#).
- em_sp_avg*: [1.2](#), 2, [2.1](#).
- em_sp_data*: 1.2, 2, 2.1.
- em_temp*: [1.2](#).
- extract_output_datum*: [1.2](#).
- FALSE*: 1.2, 2.1.
- file*: 1.2.
- FILE*: [1](#).
- fill_views*: 2.
- fill_views_master*: 1.1, [1.2](#).
- fill_views_slave*: 1.1, [2.1](#).
- FLOAT*: 1.2, 2.1.
- fwpa_avg*: [1.2](#), 2, [2.1](#), [2.5](#).
- fwpa_avg_slave*: [1.2](#).
- fwpa_sp_avg*: [1.2](#), 2, [2.1](#), [2.5](#).
- fwpa_sp_avg_slave*: [1.2](#).
- geomtesta*: 1.2.
- gpi_views*: 1, [1.2](#), 2.
- half*: 1.2.
- halfwidth*: [1.2](#), 2, [2.1](#).
- icount*: [2.7](#).
- idim*: [2.7](#).
- implicit_none_f77*: 1.1, 1.2, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7.
- implicit_none_f90*: 1.1, 1.2, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7.
- ind_tmp*: [1.2](#).
- index*: 1.2.
- index_parameters*: [1.2](#).
- int_undef*: 1.2, 2.
- int_unused*: 1.2, 2.1.
- inverted_iy*: [1.2](#), 2, [2.1](#).
- irank*: [2.7](#).
- iret*: [2.7](#).
- is*: [1.2](#), 2, [2.1](#).
- italy*: [1.2](#).
- iview*: [1.2](#), 2, [2.1](#).
- ix*: [1.2](#), 2, [2.1](#), [2.6](#), [2.7](#).
- ixprime*: [2.6](#).
- iy*: [1.2](#), 2, [2.1](#), [2.6](#), [2.7](#).
- iy_peak*: [1.2](#).
- iyprime*: [2.6](#).
- j*: [1.2](#).
- kcount*: [2.6](#), [2.7](#).
- kfilno*: [2.6](#), [2.7](#).

kx: [2.6](#), [2.7](#).
kxmin: [2.6](#), [2.7](#).
ky: [2.6](#), [2.7](#).
kymin: [2.6](#), [2.7](#).
len: [1.2](#).
loop: [2.1](#).
max: [1.2](#), [2.7](#).
max_opt: [1.2](#), [2](#), [2.1](#).
min: [1.2](#), [2.7](#).
min_dist: [1.2](#), [2](#), [2.1](#).
mod: [2](#).
mp_common: [1.1](#), [1.2](#), [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
mp_decls: [1.1](#), [1.2](#), [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
MPI: [1.2](#), [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
MPI_ANY_SOURCE: [2.3](#), [2.5](#).
MPI_bcast: [1.2](#), [2.1](#).
MPI_COMM_WORLD: [1.2](#), [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
mpi_degas2_root: [1.2](#), [2.1](#).
mpi_end: [1.1](#).
mpi_err: [1.2](#), [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
mpi_init: [1.1](#).
mpi_int: [1.2](#), [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
mpi_master: [1.1](#).
MPI_messages: [1](#), [1.2](#).
mpi_real: [1.2](#), [2.1](#), [2.5](#).
MPI_recv: [2.1](#), [2.3](#), [2.5](#).
MPI_send: [2.1](#), [2.2](#), [2.4](#).
mpi_size: [1.2](#).
MPI_SOURCE: [2.3](#).
mpi_status: [2.1](#), [2.3](#), [2.5](#).
n_avg: [1.2](#), [2](#), [2.1](#).
n_eff1: [1.2](#).
n_eff2: [1.2](#), [2](#), [2.1](#), [2.5](#).
n_eff2_slave: [1.2](#).
n_eff2_sum: [1.2](#), [2](#), [2.1](#).
n_eff3: [1.2](#).
n_sp_eff1: [1.2](#).
n_sp_eff3: [1.2](#).
name_clean: [1.2](#).
nc_read_output: [1.1](#).
nfrag: [1.2](#).
nparcel: [1.2](#).
ns: [1.2](#), [2](#), [2.1](#).
nsd: [1.2](#), [2.1](#), [2.5](#).
nslaves: [1.2](#).
nstart: [1.2](#), [2](#), [2.1](#), [2.2](#).
NSTX: [1.2](#), [2](#), [2.6](#).
NSTX_2010: [1.2](#), [2](#).
num: [1.2](#), [2](#), [2.1](#), [2.2](#).
nviews: [1.2](#).
nx: [1.2](#), [2](#), [2.1](#), [2.5](#), [2.6](#).
nx_peak: [1.2](#).
ny: [1.2](#), [2](#), [2.1](#), [2.5](#), [2.6](#).
o_mean: [1.2](#).
one: [2](#).
opt: [1.2](#), [2](#), [2.1](#).
ou_common: [1.2](#).
out_post_all: [1.2](#).
pb2dat: [2.6](#), [2.7](#).
pb2prime: [2.6](#).
peak: [1.2](#).
phi: [1.2](#), [2](#), [2.1](#).
PI: [2](#).
pmimatread: [1.1](#).
points: [1.2](#), [2](#), [2.1](#).
postdetector: [1.1](#).
pr_common: [1.2](#).
pr_test: [1.2](#).
pr_test_num: [1.2](#).
profile: [1.2](#).
readfilenames: [1.1](#).
real_unused: [1.2](#), [2.1](#).
rowflip: [2.6](#).
s: [1.2](#).
send_data: [1.2](#), [2.1](#), [2.4](#).
signal: [1.2](#), [2](#), [2.1](#).
signal_array: [1.2](#), [2](#), [2.1](#), [2.5](#).
signal_array_slave: [1.2](#).
signal_sp: [1.2](#), [2](#), [2.1](#).
signal_sp_array: [1.2](#), [2](#), [2.1](#), [2.5](#).
signal_sp_array_slave: [1.2](#).
sin: [2](#).
single: [2.7](#).
single_precision: [2.7](#).
slave: [1.2](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
slave_receive_data: [1.2](#), [2.5](#).
slave_receive_flag: [1.2](#), [2.3](#).
slave_send_flag: [1.2](#), [2.4](#).
slave_send_views: [1.2](#), [2.2](#).
sp: [1.2](#).
SP: [1.2](#).
sp_common: [1.2](#).
sp_sy: [1.2](#).
sp_sy_len: [1.2](#).
species_ind: [1.2](#), [2.1](#).
species_tally: [1.2](#).
sqrt: [1.2](#), [2](#).
st_decls: [1.2](#).
status: [1.2](#).
stderr: [1.2](#).
string_length: [1.2](#).

tag: [2.1](#), [2.2](#), [2.3](#), [2.4](#), [2.5](#).
tally_name: [1.2](#).
target_map: [1.2](#), [2](#), [2.1](#), [2.5](#).
target_map_slave: [1.2](#).
target_n: [1.2](#).
target_ne: [1.2](#).
target_r: [1.2](#).
target_te: [1.2](#).
target_z: [1.2](#).
temp_array: [1.2](#).
test: [1.2](#).
test_density: [1.2](#), [2](#), [2.1](#).
tl_check: [1.2](#).
tl_common: [1.2](#).
tl_index_max: [1.2](#).
tl_index_test: [1.2](#).
tl_index_zone: [1.2](#).
tl_num: [1.2](#).
total_tally: [1.2](#).
transpose: [2.6](#).
trim: [1.2](#).
TRUE: [1.2](#), [2.1](#), [2.3](#).
unit: [1.2](#).
var_alloc: [1.2](#), [2.1](#).
var_free: [1.2](#), [2.1](#).
vc_args: [2](#).
vc_decl: [1.2](#), [2.1](#).
vc_decls: [1.2](#), [2.1](#).
w_per_atom: [1.2](#).
w_sp_per_atom: [1.2](#).
web: [1.1](#), [1.2](#).
wrdata: [2.6](#), [2.7](#).
wrdatap: [1.2](#), [2.6](#).
x_array: [1.2](#).
xdat: [2.6](#), [2.7](#).
xerrab: [2.7](#).
xprime: [2.6](#).
y_array: [1.2](#).
ydat: [2.6](#), [2.7](#).
yprime: [2.6](#).
zdata: [2.7](#).
zdati: [2.7](#).
zero: [1.2](#), [2](#).
zmax: [2.7](#).
zmin: [2.7](#).
zn_check: [1.2](#).
zn_common: [1.2](#), [2.1](#).
zn_num: [1.2](#), [2](#), [2.1](#).
zn_plasma: [1.2](#).
zn_type: [1.2](#).
zn_volume: [1.2](#), [2](#).
zone: [1.2](#), [2](#), [2.1](#).
zone_center: [1.2](#), [2](#).
zone_frgs: [1.2](#), [2](#), [2.1](#).
zone_ind: [1.2](#), [2.1](#).
zxd: [2.7](#).
zydat: [2.7](#).

〈Compute Signal 2〉 Used in sections 1.2 and 2.1.

〈Functions and Subroutines 1.2, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7〉 Used in section 1.1.

〈Memory allocation interface 0〉 Used in sections 2.5, 2.4, 2.3, 2.2, 2.1, and 1.2.

COMMAND LINE: "fweave -f -i! -W[-ykw800 -ytw40000 -j -n/
/u/dstotler/degas2/src/pstdetector.web".

WEB FILE: "/u/dstotler/degas2/src/pstdetector.web".

CHANGE FILE: (none).

GLOBAL LANGUAGE: FORTRAN.